

US-08: Account Email Verification

AI Prompt / Reference Log

Prompt / Request

Implement US-08: Account Email Verification in the existing CareerConnect project while maintaining the current project structure, coding style, authentication system, database schema, and UI design.

The implementation should use the existing SvelteKit, Better Auth, Drizzle ORM, Supabase/PostgreSQL, and Tailwind CSS setup.

- Send a verification email after registration.
- Reuse Better Auth's existing emailVerified field and verification table.
- Prevent unverified users from accessing protected application pages.
- Allow verification links to activate the account.
- Add a resend-verification-email option.
- Redirect newly registered users to a page informing them that a verification email has been sent.
- Inform users that the verification link expires after one hour.
- Redirect verified users back to the login flow.
- Keep all new pages consistent with the existing CareerConnect authentication UI.
- Add automated tests for verified and unverified account access.
- Avoid unnecessary database migrations, duplicate verification fields, or unrelated refactoring.

Development and Understanding Process

The implementation was completed step by step rather than applying all changes at once.

Throughout the process, the user actively questioned the suggested implementation and asked for explanations of how the code worked before continuing to the next step.

- How Better Auth stores the emailVerified value.
- Why no new verification field or database table was required.
- The difference between Better Auth's sendVerificationEmail API and the custom email-sending function.
- Why requireEmailVerification and autoSignIn were needed.
- How the registration redirect works.
- How the verification callback URL works.
- Why the verification and authentication routes must remain public.
- How the SvelteKit hook blocks unverified users.
- How the resend-verification functionality works.
- How the login page detects a successfully verified account.

- Where new files and test files should be placed in the project.
- How the automated tests verify guest, unverified, and verified access.

The user reviewed each modification, ran the project checks after the changes, reported errors when they appeared, and continued only after understanding and confirming the previous step.

Main Implementation Changes

The existing Better Auth schema was first checked and already contained the required emailVerified field and verification table. Therefore, no additional verification database schema or migration was added.

Better Auth was configured to require email verification, send a verification email after signup, prevent automatic login before verification, prevent automatic login immediately after verification, and use a one-hour verification-link expiration time.

A server-side email utility was added using Resend to send the verification link generated by Better Auth.

The registration flow was updated so that after account creation the user is redirected to a verification confirmation page instead of directly entering the application.

A verification-sent page was added using the project's existing authentication layout. It tells the user that a verification email has been sent, that the verification link expires in one hour, and that the user must verify the email before logging in.

A verification page with a resend option was also added.

The login flow was updated to display a successful verification message when the user returns after verifying the email.

The existing SvelteKit authentication hook was updated to distinguish between guests, logged-in but unverified users, and verified users. Unverified users are redirected to the email-verification flow, while verified users can access protected application routes.

Automated tests were added for public authentication routes, guest access to protected routes, unverified-user access, and verified-user access.

User Testing and Verification

The user personally configured the required Resend environment variables and tested the email-verification flow using a real email account.

- Registration successfully sends a verification email.
- The verification confirmation page appears after registration.
- The verification link is received by email.
- The verification link successfully activates the account.
- The email_verified value in Supabase changes to true after verification.
- Verified users can log in and access the application.
- Unverified users are prevented from accessing protected pages.
- The resend-verification functionality works.

- The verification-success message appears during the login flow.
- pnpm check completed with 0 errors and 0 warnings.
- pnpm test completed with all four email-verification access tests passing.

Final Result

US-08 was completed using the existing Better Auth authentication and database structure without introducing duplicate verification fields or unnecessary database migrations.

Task	Status
8.1 Send verification email after registration	Completed
8.2 Store whether the user's email is verified	Completed
8.3 Block application access for unverified users	Completed
8.4 Allow verification links to activate accounts	Completed
8.5 Add resend verification option	Completed
8.6 Test verified and unverified account access	Completed